МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №4

з курсу: «Хмарні технології та сервіси»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Хід роботи

1. Monitoring Setup

1.1 Set up metrics collection (Prometheus latency (p50, p95, p99), throughput, error rate for coordinator and shards

Виконаємо реалізацію збору метрик з координатора та шардів за допомогою системи моніторингу. Вимірюються ключові показники продуктивності: latency запитів (p50, p95, p99), пропускна здатність (throughput) та рівень помилок (error rate). Метрики збираються окремо для координатора та кожного шарда, що дозволяє оцінити стан системи та виявляти вузькі місця.

```
HTTP_REQUESTS_TOTAL = Counter(
    name: "http_requests_total",
    documentation: "Total HTTP requests",
    labelnames: ["service", "method", "endpoint", "status"]
)

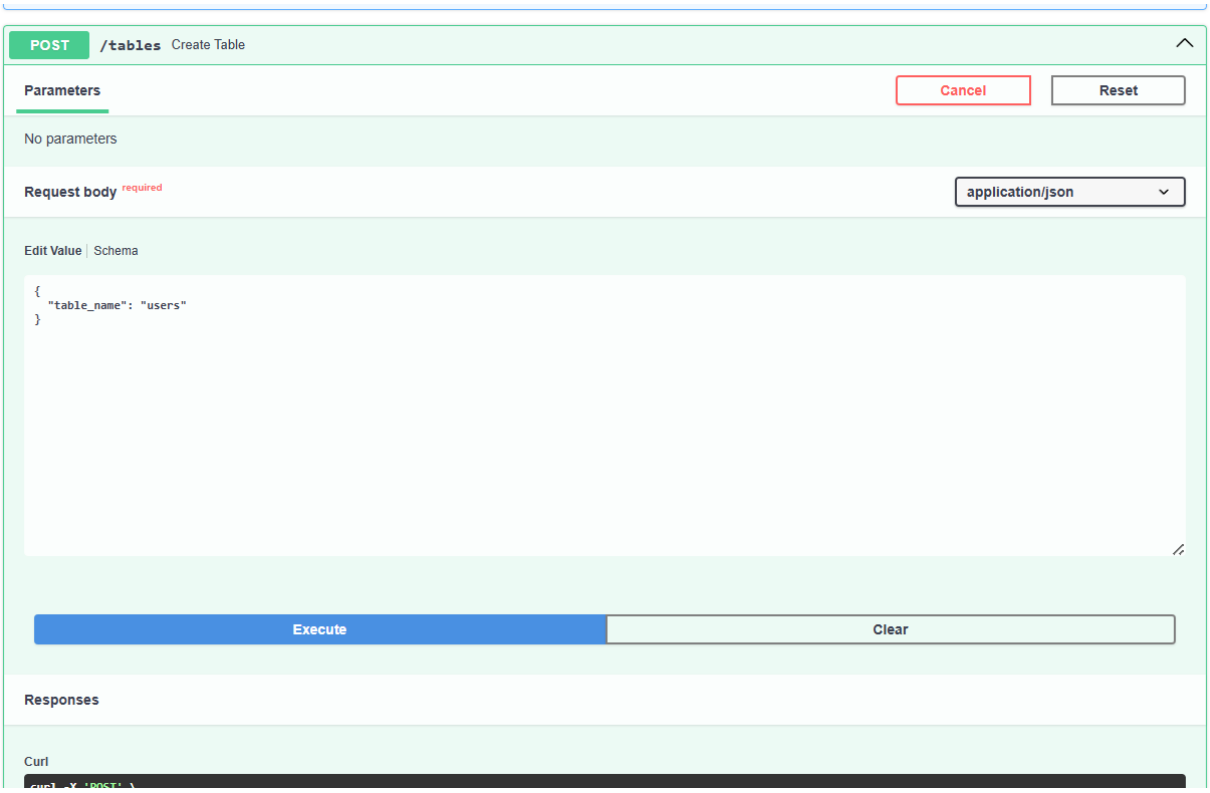
HTTP_ERRORS_TOTAL = Counter(
    name: "http_errors_total",
    documentation: "Total HTTP errors",
    labelnames: ["service", "endpoint", "status_class"]
)

HTTP_REQUEST_DURATION = Histogram(
    name: "http_request_duration_seconds",
    documentation: "HTTP request latency",
    labelnames: ["service", "method", "endpoint"],
```

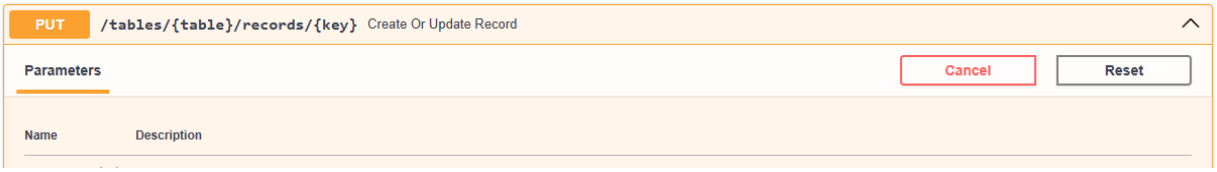
Даний модуль відповідає за збір метрик кількості запитів, помилок та часу обробки, що дозволяє обчислювати p50, p95 та p99 latency.

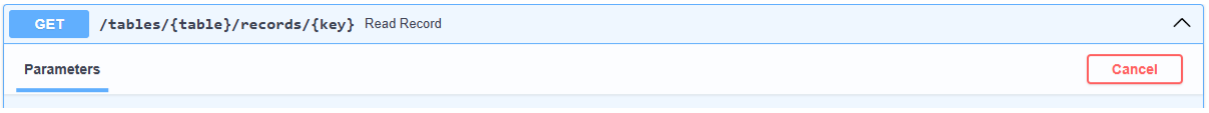
```
/metrics
@app.get("/metrics")
def metrics():
    return Response(
        generate_latest(),
        media_type=CONTENT_TYPE_LATEST
    )
```

Спочатку перевіремо доступність Coordinator-сервісу через Swagger UI. Coordinator виступає єдиною точкою входу до розподіленої системи та відповідає за маршрутизацію запитів до відповідних shard-вузлів + створемо логічну таблицю users через Coordinator. Таблиця не прив’язана до конкретного shard-вузла, а використовується як абстракція для подальшого шардінгу записів за ключем



Виконаємо PUT та GET запити щоб згенерувати трафік (метрики latency, throughput, error rate будуть порожні в іншому випадку)





Для перевірки стабільності роботи системи було перевірено endpoint /metrics, який експортує технічні метрики для Prometheus, включаючи кількість запитів, час відповіді та помилки.

GET /metrics Metrics

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/metrics' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/metrics
```

Server response

Code Details

200

Response body

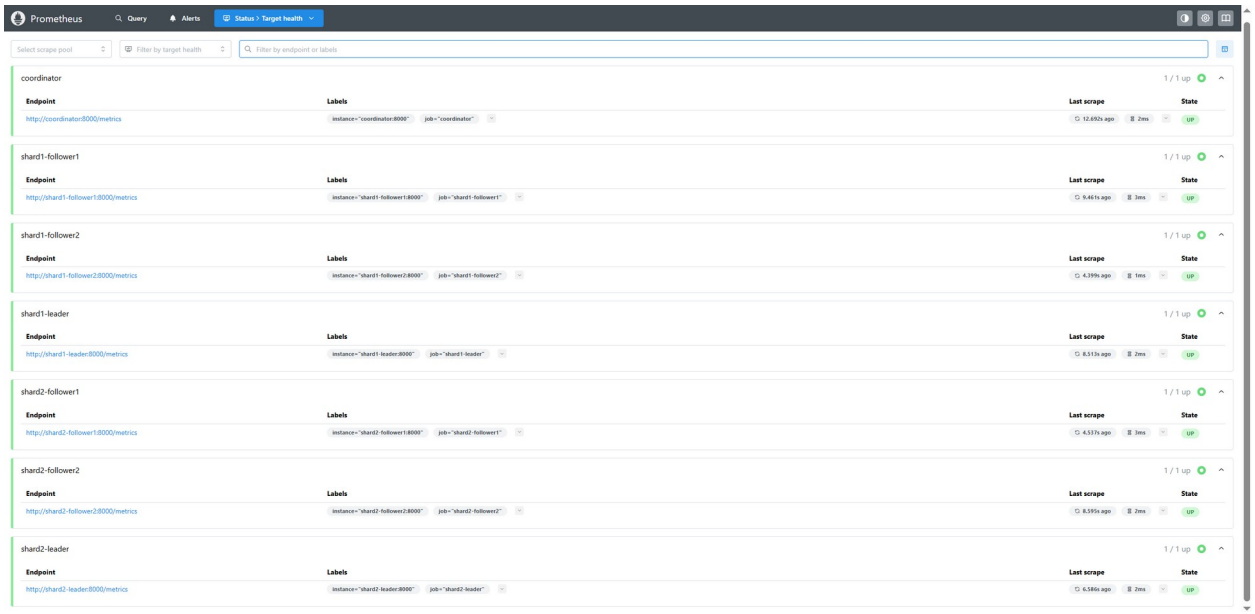
```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 808.0
python_gc_objects_collected_total{generation="1"} 229.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 167.0
python_gc_collections_total{generation="1"} 15.0
python_gc_collections_total{generation="2"} 1.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="14",version="3.11.14"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 3.95440128e+08
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 5.6692736e+07
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.76576141009e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
```

Download

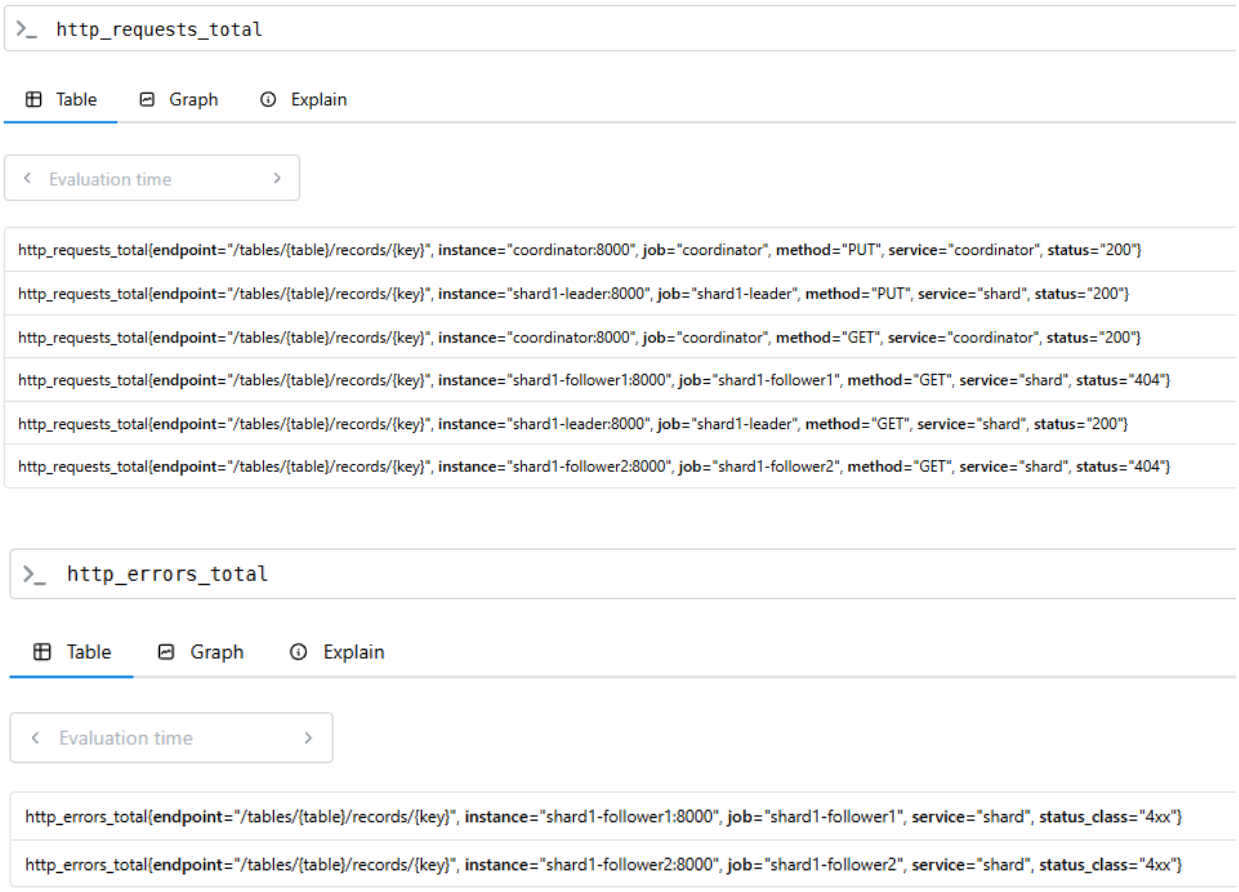
Response headers

```
content-length: 7991
content-type: text/plain; version=1.0.0; charset=utf-8
date: Mon,15 Dec 2025 03:56:24 GMT
server: uvicorn
x-trace-id: 430156f1-17ec-4a00-9c59-27817da75153
```

Перейдемо у Prometheus, та перевіримо статус Endpoint

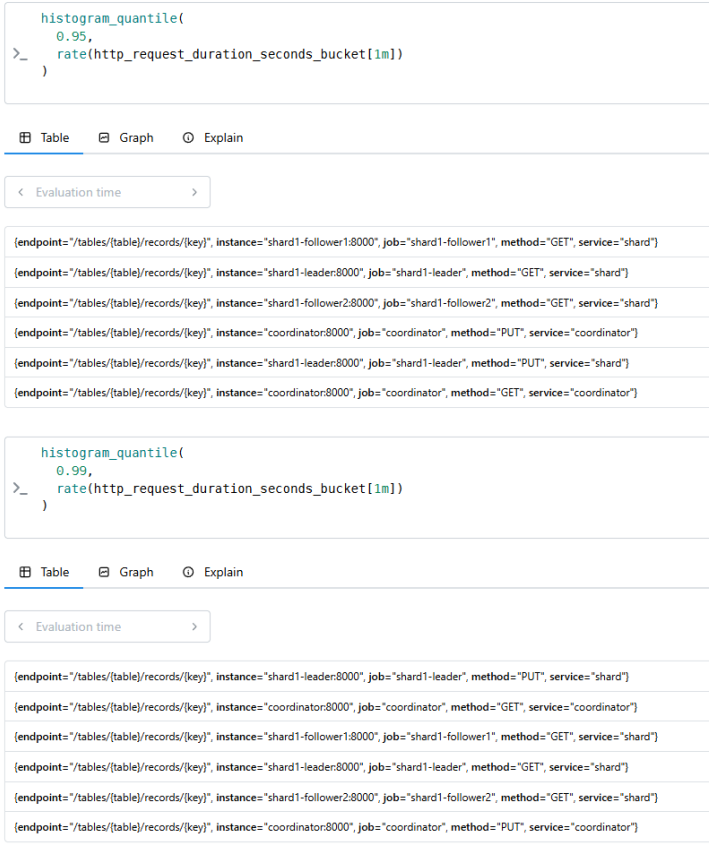


Виконаємо http запити



Попередньо ми виконали запити типу PUT та GET для перевірки працездатності координатора та shard-вузлів. Метрика http_requests_total відображає загальну кількість оброблених HTTP-запитів з розбиттям за endpoint, методом, сервісом та кодом відповіді. З результатів видно, що координатор успішно обробив PUT і GET запити зі статусом 200, а shard-вузли також отримували відповідні запити. Наявність записів зі статусом 404 на follower-вузлах є очікуваною поведінкою, оскільки читання виконується з лідера, а followers не обслуговують read-запити напряму.

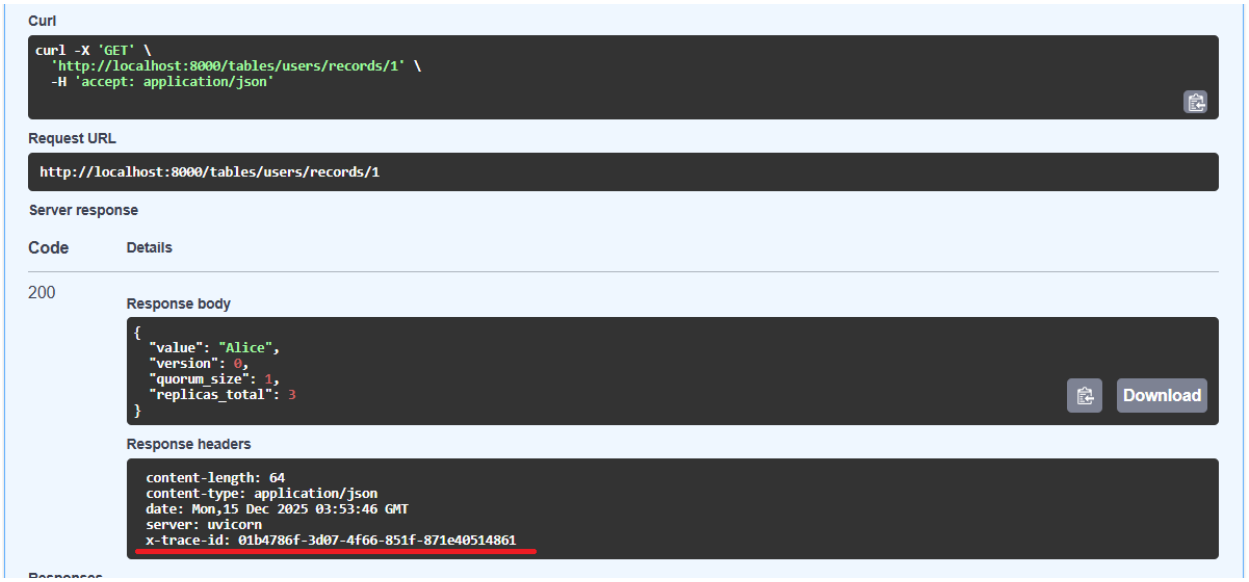
Виконаємо запит



Для оцінки продуктивності системи було виконано запит histogram quantile (0.95, rate(http_request_duration_seconds_bucket[1m])), який призначений для обчислення 95-го перцентиля затримки обробки HTTP-запитів. Отримані значення (NaN) пояснюється недостатньою кількістю запитів у вибірці за заданий часовий інтервал (очікувано для тестового навантаження і підтверджує коректність збору метрик затримки без викривлення результатів)

1.2 datadog + structured logging with trace IDs to track requests through the system.

Реалізуємо трасування запитів у розподіленій системі через trace ID. Координатор буде генерувати унікальний trace_id для кожного HTTP-запиту, додавати його в логи та передавати до shard-вузлів через заголовок X-Trace-Id. Це дозволяє швидко знаходити всі події, пов’язані з одним запитом, навіть коли він проходить через кілька сервісів



Structured logging реалізовано через кореляційний trace_id та структуровані метрики Prometheus, де кожен запит має атрибути endpoint, method, service, status та instance.

Datalog можна побачити (так як я працюю з докером) прописавши команди docker logs coordinator (або напр. docker logs shard1-leader)
Координатора:

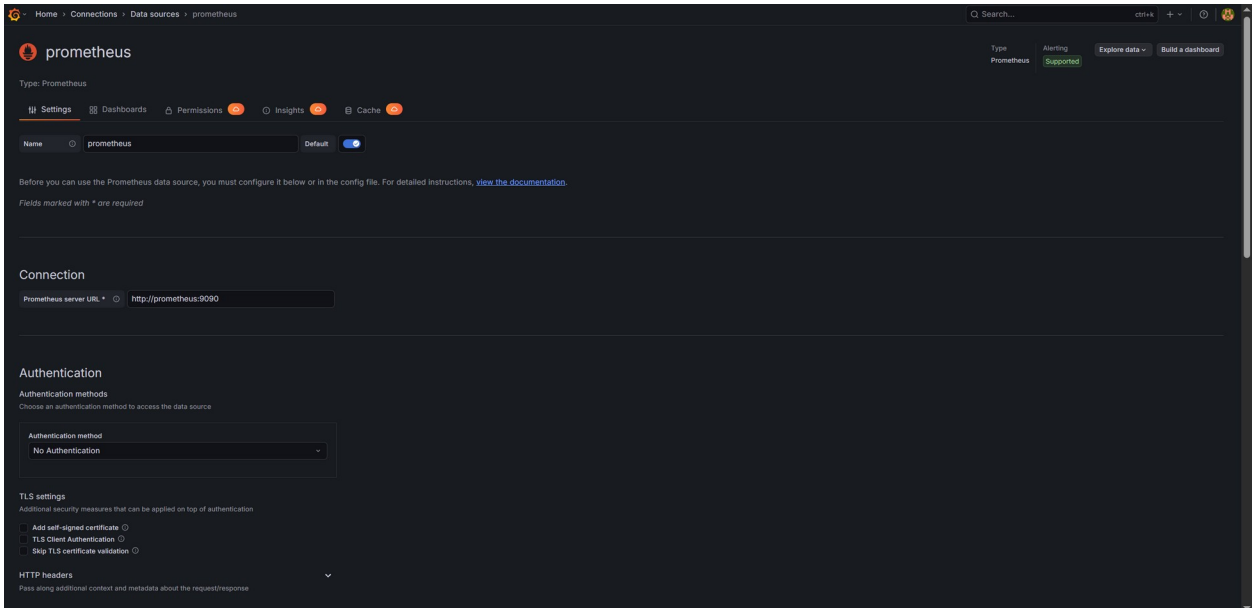
```
PS C:\Users\user\PycharmProjects\coordinator> docker logs coordinator
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     172.20.0.2:49330 - "POST /shards/register HTTP/1.1" 200 OK
INFO:     172.20.0.4:45226 - "POST /shards/register HTTP/1.1" 200 OK
INFO:     172.20.0.3:47736 - "POST /shards/register HTTP/1.1" 200 OK
INFO:     172.20.0.7:35596 - "POST /shards/register HTTP/1.1" 200 OK
INFO:     172.20.0.6:34756 - "POST /shards/register HTTP/1.1" 200 OK
INFO:     172.20.0.5:43692 - "POST /shards/register HTTP/1.1" 200 OK
INFO:     172.20.0.9:56510 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:43642 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.1:41566 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.20.0.1:41566 - "GET /openapi.json HTTP/1.1" 200 OK
```

Шард лідера

```
PS C:\Users\user\PycharmProjects\coordinator> docker logs shard1-leader
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     172.20.0.9:48114 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:38888 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:60470 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:54678 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:49288 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:60622 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:58972 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:36060 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:47100 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.20.0.9:40134 - "GET /metrics HTTP/1.1" 200 OK
```

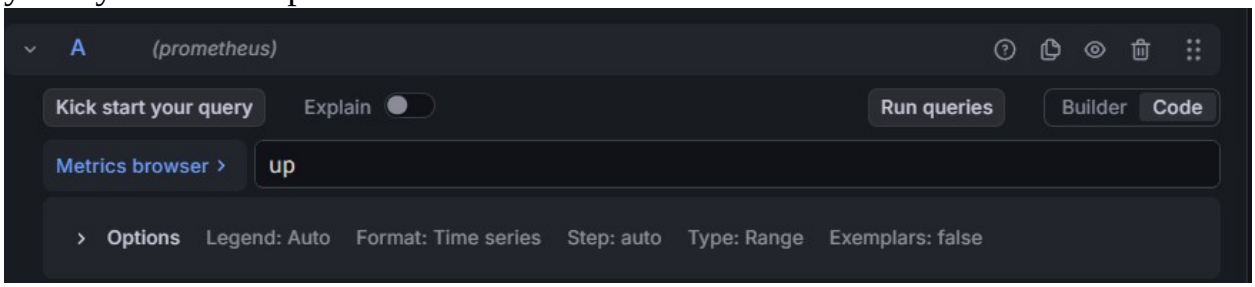
1.3 Create a dashboard with key metrics: cluster health, replication lag, shard distribution.

Створемо дашборд у Grafana для візуалізації ключових метрик роботи розподіленого сховища даних. Джерело даних – Prometheus (збирає метрики з coordinator та shard-вузлів через HTTP endpoint /metrics)



Моніторинг стану кластера (Cluster Health)

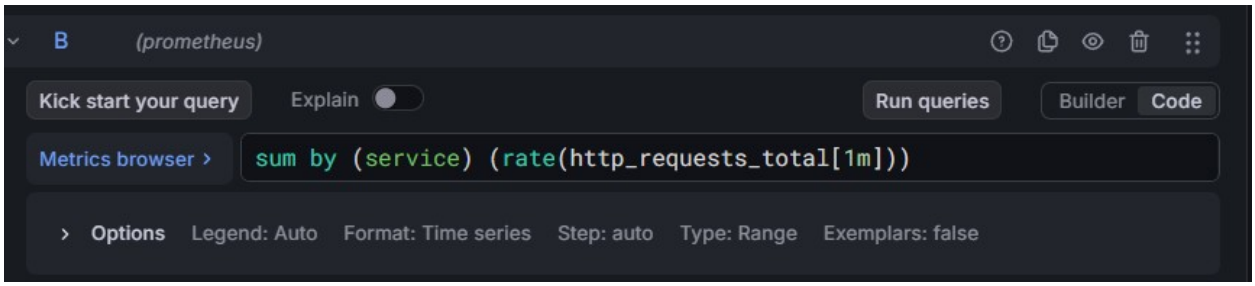
Для оцінки доступності компонентів системи використано метрику up. Значення up = 1 свідчить про те, що відповідний сервіс є активним та доступним. На дашборді відображено стан координатора, лідерів та реплік шардів, що дозволяє в режимі реального часу контролювати працездатність усіх вузлів кластера.



Розподіл навантаження між вузлами (Shard Distribution / Throughput)

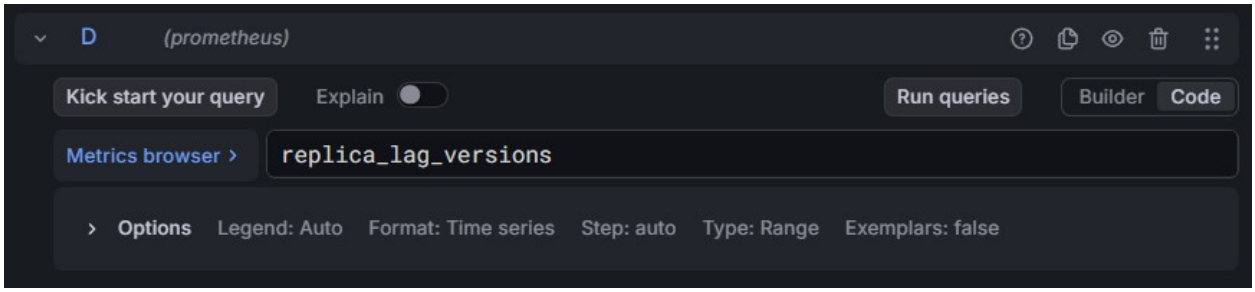
Для аналізу інтенсивності обробки запитів використано метрику http_requests_total разом із функцією rate, що дозволяє визначити кількість HTTP-запитів за одиницю часу. Агрегація за сервісами та інстансами демонструє, як навантаження розподіляється між coordinator та shard-вузлами

під час виконання операцій PUT та GET, підтверджуючи коректну маршрутизацію запитів у системі.

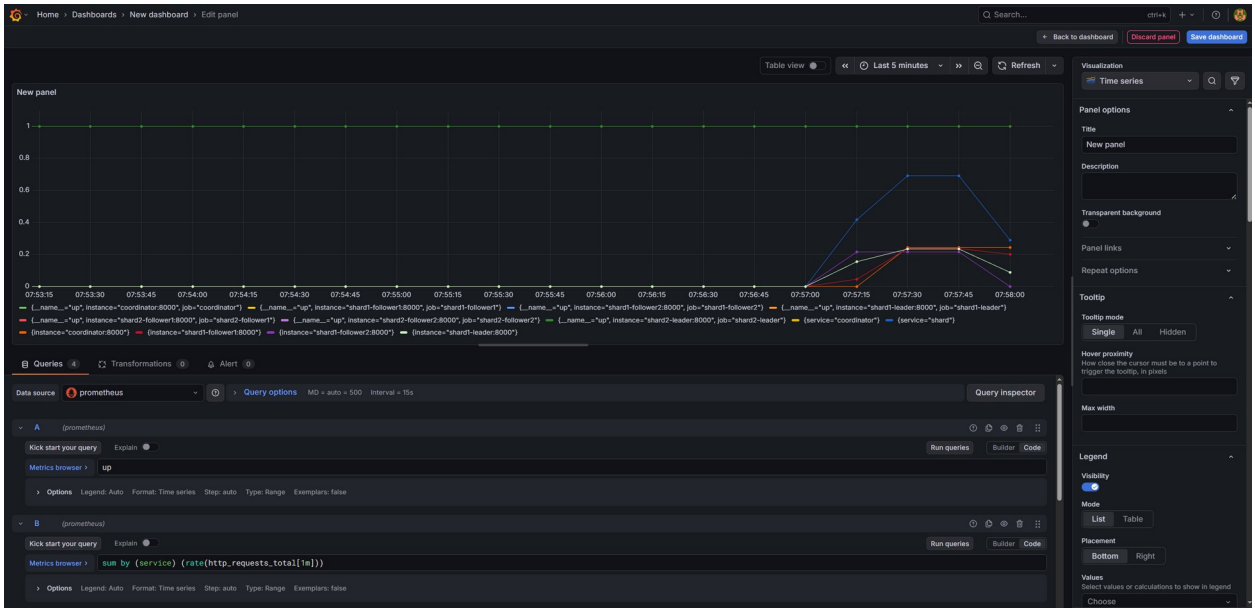


Затримка реплікації (Replication Lag)

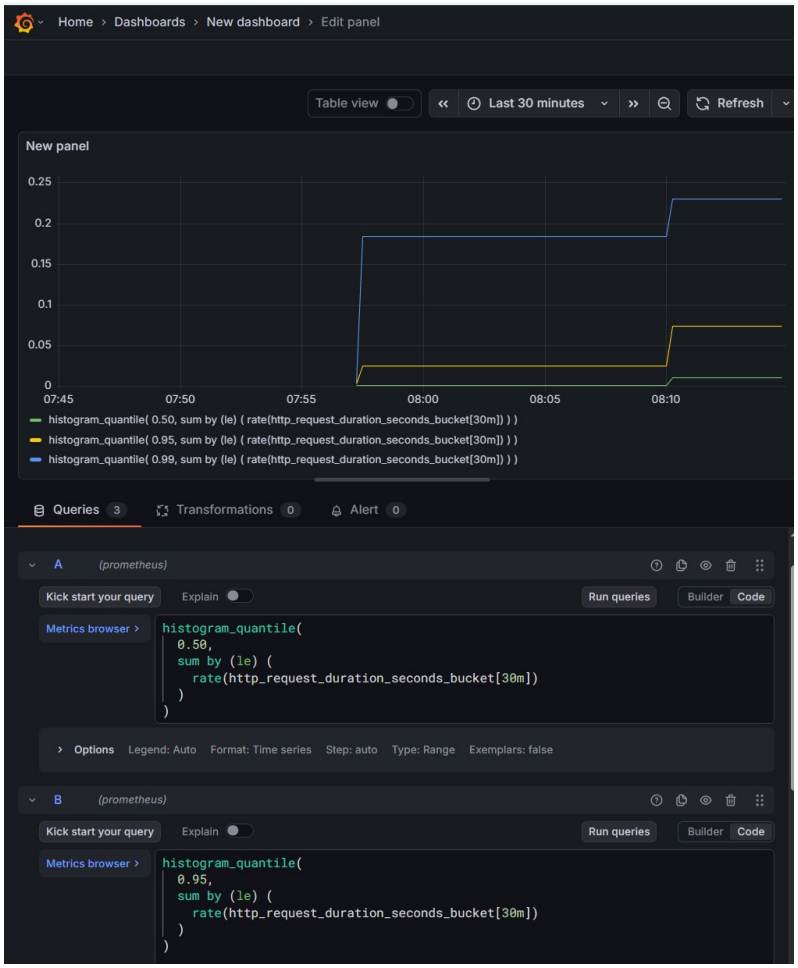
Для контролю узгодженості даних між лідером та репліками використано метрику replica_lag_versions, яка відображає різницю версій даних між основним шардом та його репліками. Нульове значення цієї метрики свідчить про відсутність затримки реплікації та синхронний стан реплік, що підтверджує коректну роботу механізму реплікації.



У результаті маємо:



На дашборді відображено стан кластера та інтенсивність обробки запитів. Метрика up підтверджує доступність усіх компонентів системи (coordinator, shard leaders, shard followers). Метрика rate(http_requests_total) демонструє реальний HTTP-трафік, який з’являється під час виконання PUT та GET запитів, що підтверджує коректну маршрутизацію та розподіл навантаження між вузлами розподіленої системи.



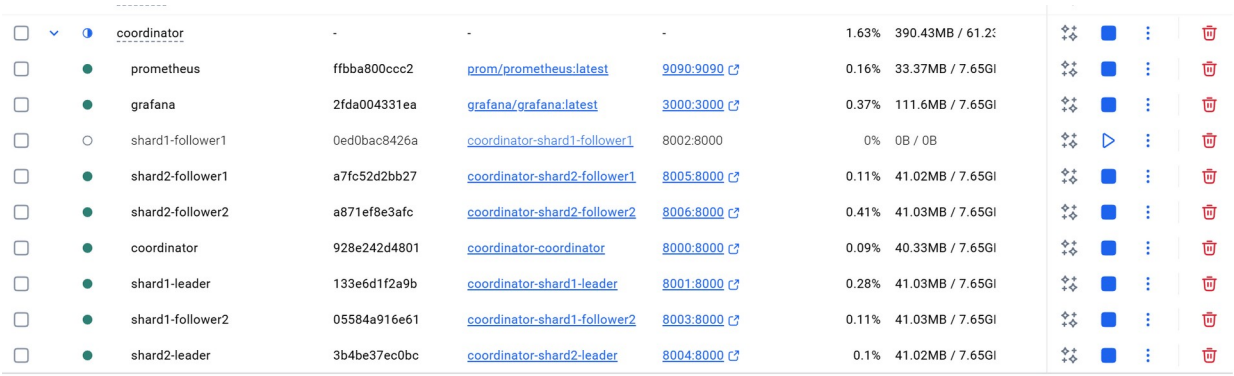
На панелі відображено percentiles латентності HTTP-запитів (p50, p95, p99), обчислені на основі гістограм Prometheus

1. p50 ~ 0–5 ms більшість запитів швидкі
2. p95 ~ 25 ms
3. p99 ~ 180–230 ms поодинокі повільні запити

Такий вид графіків можна пояснити нерівномірним надходженням запитів та дискретною природою обчислення percentile з гістограм. Показник p50 залишається на низькому рівні (швидка обробка більшості запитів) p99 демонструє поодинокі повільні відповіді, характерні для tail latency.

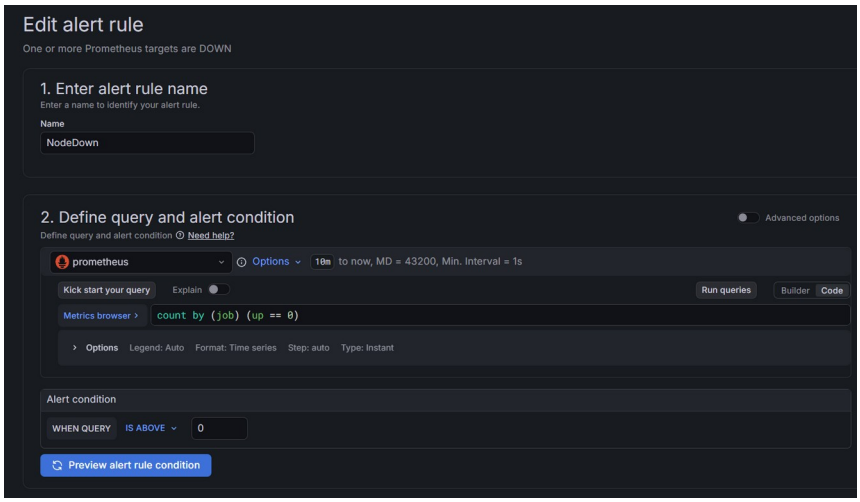
1.4 Set up alerting for critical events: node down, high latency (p99 > threshold), replication lag > N seconds.

Node Down. Для реалізації спочатку зупинемо одну ноду:

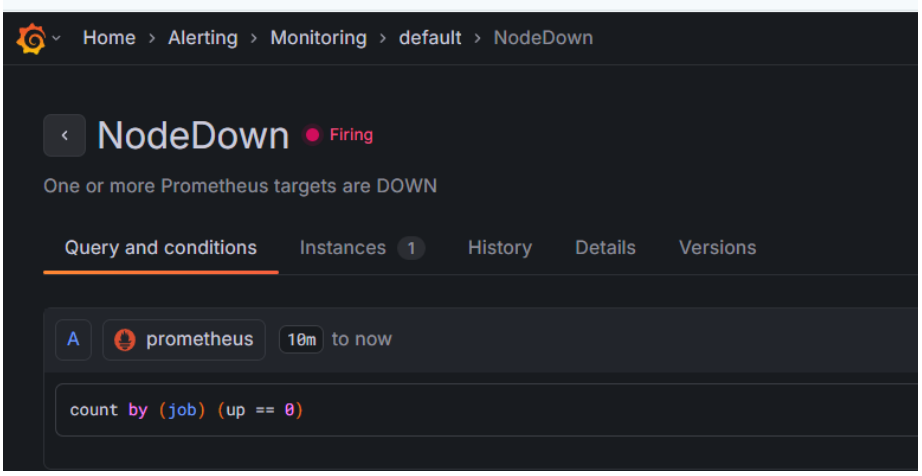


	coordinator	-	-	-	1.63%	390.43MB / 61.2	
	prometheus	ffbba800ccc2	prom/prometheus:latest	9090:9090	0.16%	33.37MB / 7.65GI	
	grafana	2fda004331ea	grafana/grafana:latest	3000:3000	0.37%	111.6MB / 7.65GI	
	shard1-follower1	0ed0bac8426a	coordinator-shard1-follower1	8002:8000	0%	0B / 0B	
	shard2-follower1	a7fc52d2bb27	coordinator-shard2-follower1	8005:8000	0.11%	41.02MB / 7.65GI	
	shard2-follower2	a871ef8e3afc	coordinator-shard2-follower2	8006:8000	0.41%	41.03MB / 7.65GI	
	coordinator	928e242d4801	coordinator-coordinator	8000:8000	0.09%	40.33MB / 7.65GI	
	shard1-leader	133e6d1f2a9b	coordinator-shard1-leader	8001:8000	0.28%	41.03MB / 7.65GI	
	shard1-follower2	05584a916e61	coordinator-shard1-follower2	8003:8000	0.11%	41.03MB / 7.65GI	
	shard2-leader	3b4be37ec0bc	coordinator-shard2-leader	8004:8000	0.1%	41.02MB / 7.65GI	

Налаштуємо alert:



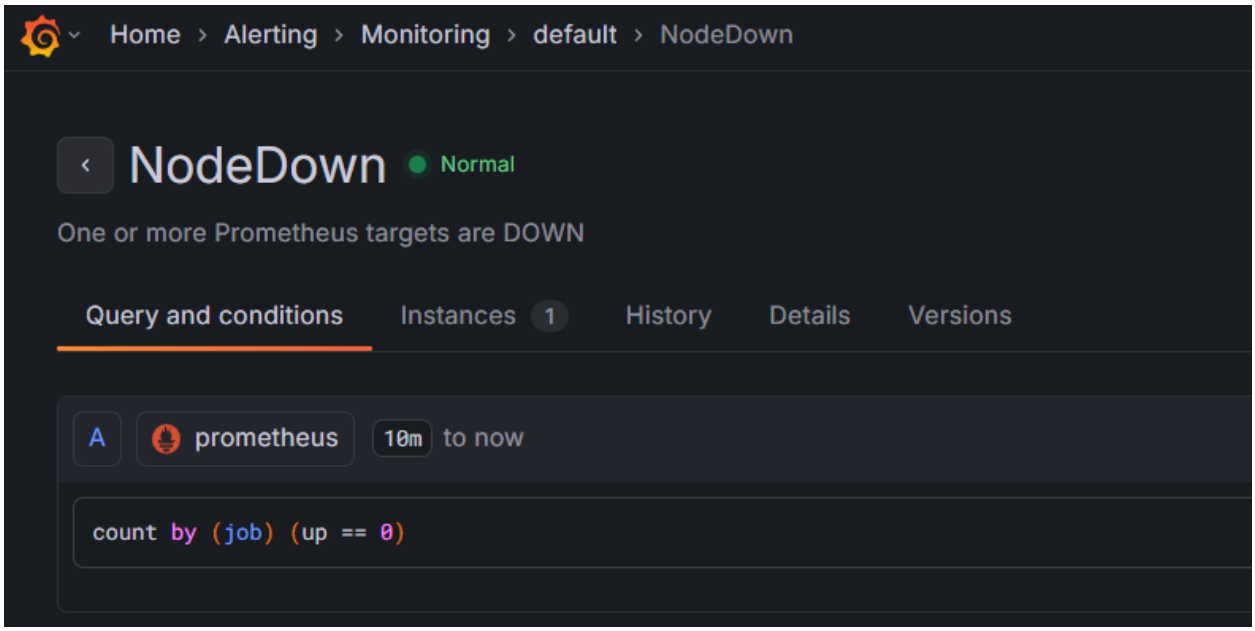
При зупинці ноди маємо:



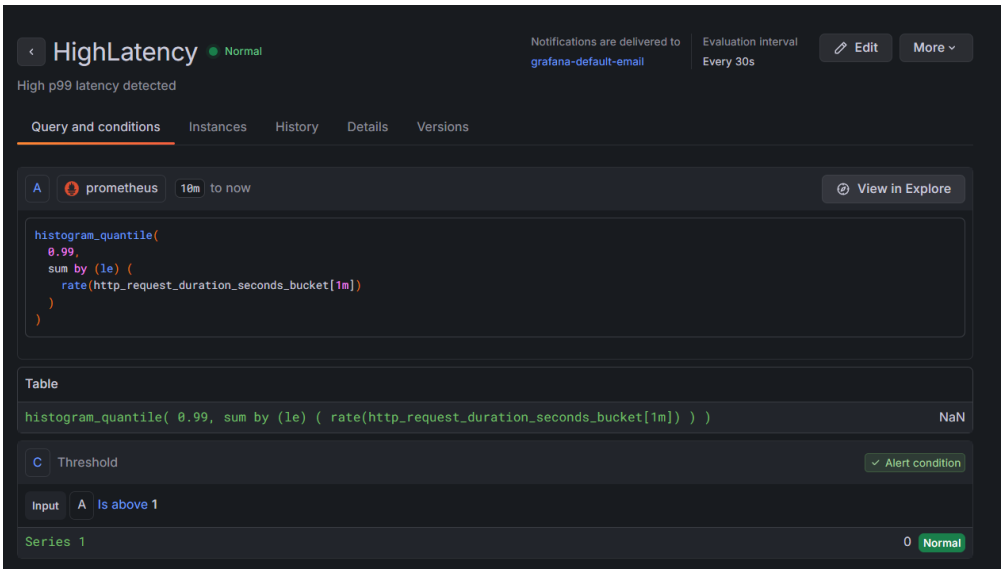
Коли все активно:

coordinator	-	-	-	1.96%	435.24MB / 68.8		
prometheus	ffbba800ccc2	prom/prometheus:latest	9090:9090	0.15%	34.02MB / 7.65GI		
grafana	2fda004331ea	grafana/grafana:latest	3000:3000	0.72%	115.2MB / 7.65GI		
shard1-follower1	0ed0bac8426a	coordinator-shard1-follower1	8002:8000	0.25%	40.56MB / 7.65GI		
shard2-follower1	a7fc52d2bb27	coordinator-shard2-follower1	8005:8000	0.12%	41.02MB / 7.65GI		
shard2-follower2	a871ef8e3afc	coordinator-shard2-follower2	8006:8000	0.15%	41.03MB / 7.65GI		
coordinator	928e242d4801	coordinator-coordinator	8000:8000	0.14%	40.33MB / 7.65GI		
shard1-leader	133e6d1f2a9b	coordinator-shard1-leader	8001:8000	0.13%	41.03MB / 7.65GI		
shard1-follower2	05584a916e61	coordinator-shard1-follower2	8003:8000	0.12%	41.03MB / 7.65GI		
shard2-leader	3b4be37ec0bc	coordinator-shard2-leader	8004:8000	0.18%	41.02MB / 7.65GI		

Маємо:

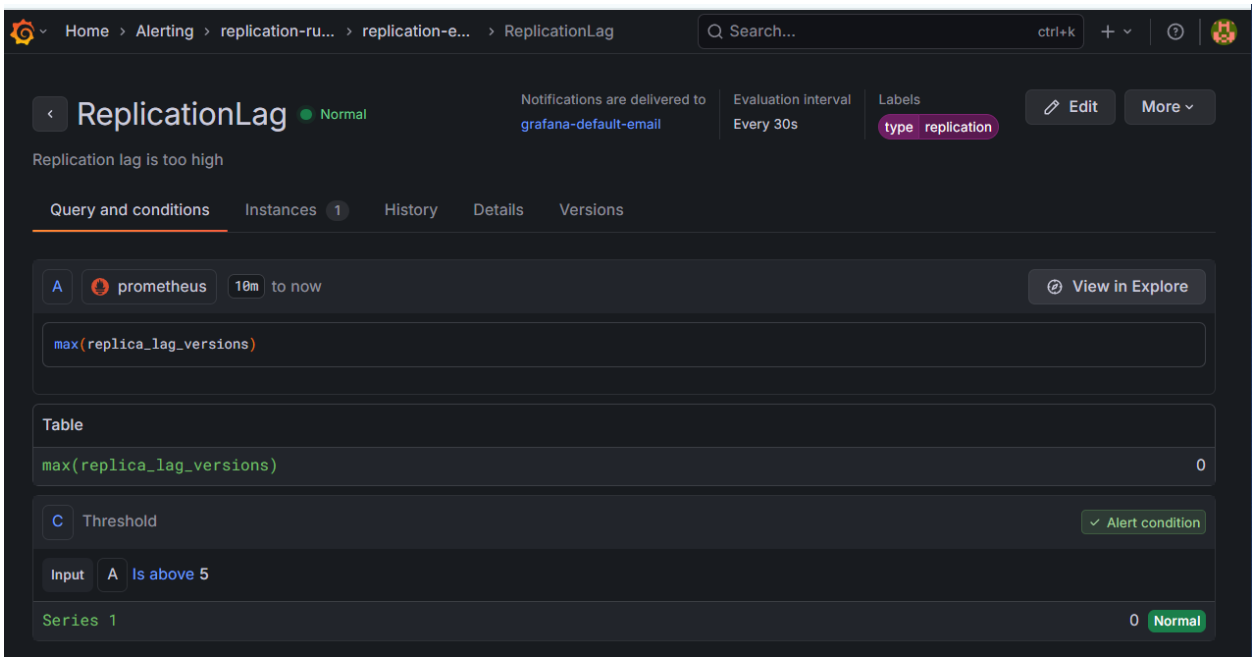


Наступний Alert це High Latency, налаштувавши його отримуємо:



При умові, якщо кожен запит ≈ 1.5s, стане p99 > 1s, і статус буде Firing, Алерт не спрацьовував, тому що фактична p99 затримка була ~100 мс, що нижче порогу 1 секунди.

Для контролю узгодженості реплік було реалізовано ReplicationLag. Алерт базується на метриці replica_lag_versions, яка відображає кількість версій, на які репліка відстає від лідера. У разі перевищення максимального лагу більше ніж на 5 версій протягом 1 хвилини алерт переходить у стан FIRING.



Алерт у стані Normal, оскільки в системі відсутні умови деградації (висока латентність або затримка реплікації), для мого випадку це очікуваний і правильний стан для стабільної системи.

2. Conflict Resolution

2.1 Implement versioning for records (vector clocks or timestamps).

Для виконання пункту потрібно додати версію до кожного запису у вигляді часової мітки (timestamp). Версія створюється на лідері під час операції запису (PUT) і передається разом із записом на всі репліки. Таким чином кожен запис має поле version, яке дозволяє порівнювати різні копії одного запису, отримані з різних реплік.

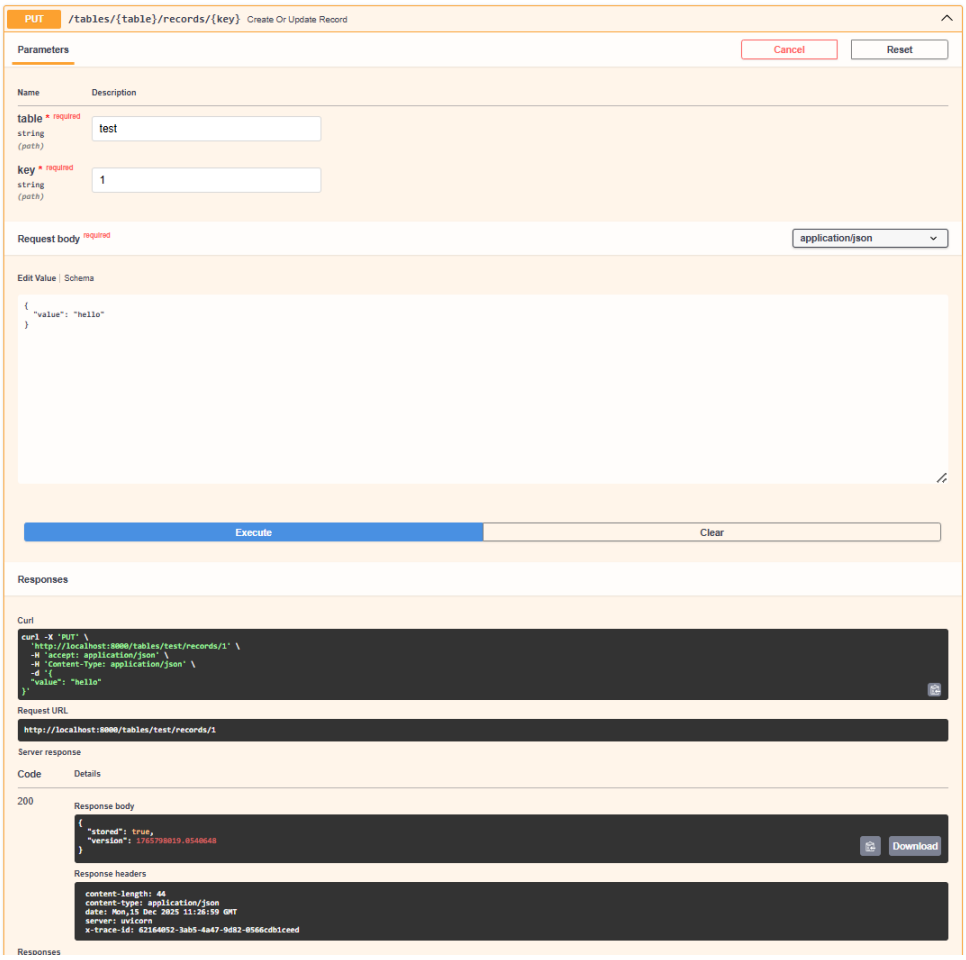
Timestamp це по суті часова мітка, яка відображає точний момент створення запису. У моїй реалізації timestamp генерується на shard-leader за допомогою функції time.time() під час операції запису. Ця версія зберігається

разом із даними та реплікується на всі копії, що дозволяє порівнювати різні версії одного запису в розподіленій системі.

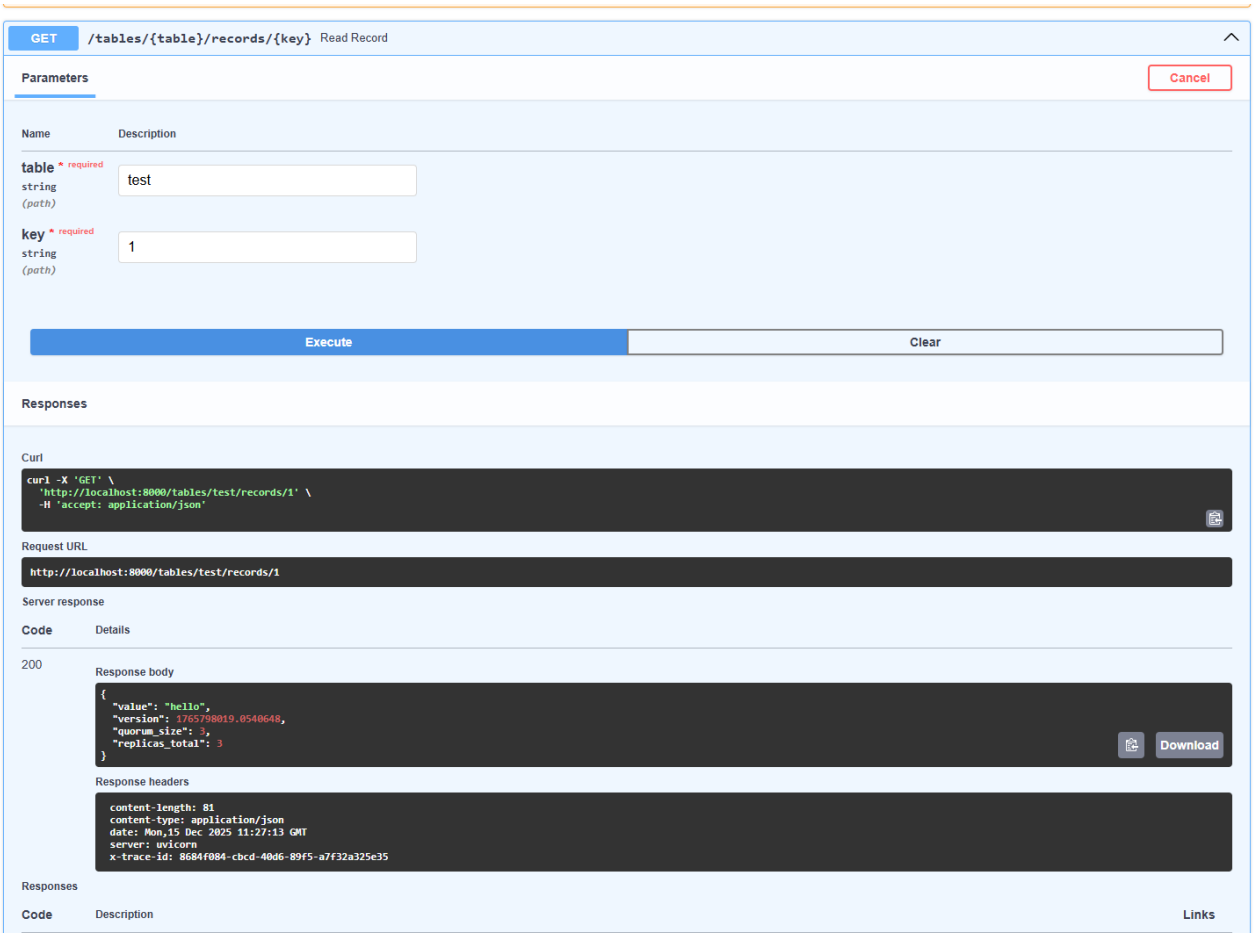
Тут реалізується механізм версіонування записів з використанням часових міток (timestamps). Під час кожної операції запису нова версія формується на shard-leader у вигляді UNIX timestamp, який відображає точний момент створення запису. Отримана версія зберігається разом із даними та реплікується на всі копії, що дозволяє надалі порівнювати різні версії одного запису та виявляти конфлікти при конкурентних оновленнях.



Під час виконання операції PUT створюється часова мітка за допомогою функції time.time(), яка повертає поточний UNIX timestamp. Значення timestamp зберігається у полі version разом із даними запису. Оскільки версія формується безпосередньо на shard-leader, вона є єдиною для всіх реплік та коректно відображає момент створення запису.



Виконання операції запису через API координатора. У відповіді сервера присутнє поле version, яке містить значення UNIX timestamp. Це підтверджує, що версія запису була згенерована автоматично під час операції запису та повернута клієнту.



У відповіді сервера повертається значення поля version, яке повністю співпадає з версією, отриманою під час операції PUT. Це підтверджує, що версія зберігається разом із даними та коректно передається між репліками під час читання.

2.2 Implement Last-Write-Wins (LWW) conflict resolution strategy.

Last-Write-Wins (LWW) стратегія автоматичного вирішення конфліктів. Суть підходу полягає в тому, що у разі отримання кількох версій одного запису система порівнює їх за полем version, яке містить часову мітку. Запис із більшою часовою міткою вважається новішим та зберігається, а застарілий запис ігнорується. Такий підхід дозволяє уникнути блокувань та забезпечує eventual consistency у розподіленій системі.

Якщо у розподіленій системі існує декілька версій одного запису, система автоматично обирає ту версію, яка має більшу часову мітку (timestamp). Таким чином, останній запис вважається актуальним, а застарілі версії ігноруються. Це дозволяє уникнути блокувань і забезпечує eventual consistency.

```python
/internal/replicate
@app.post("/internal/replicate")
def replicate(event: dict):
    global last_applied_version

    table = event["table"]
    key = event["key"]
    incoming = event["record"]

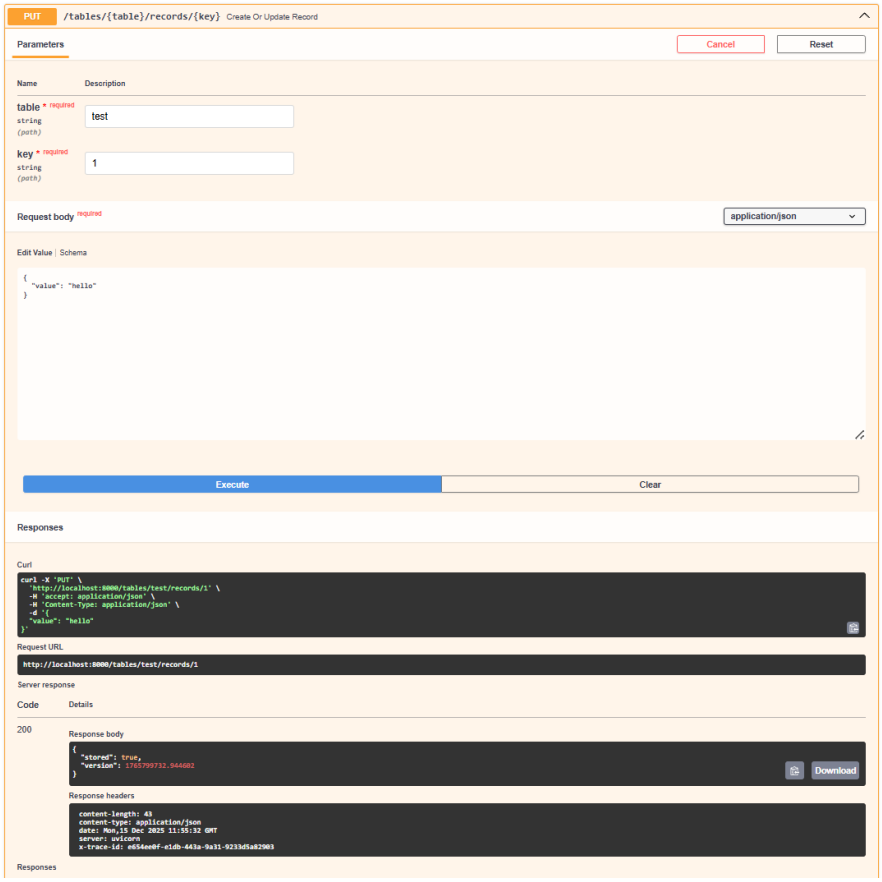
    local = storage.get(table, {}).get(key)

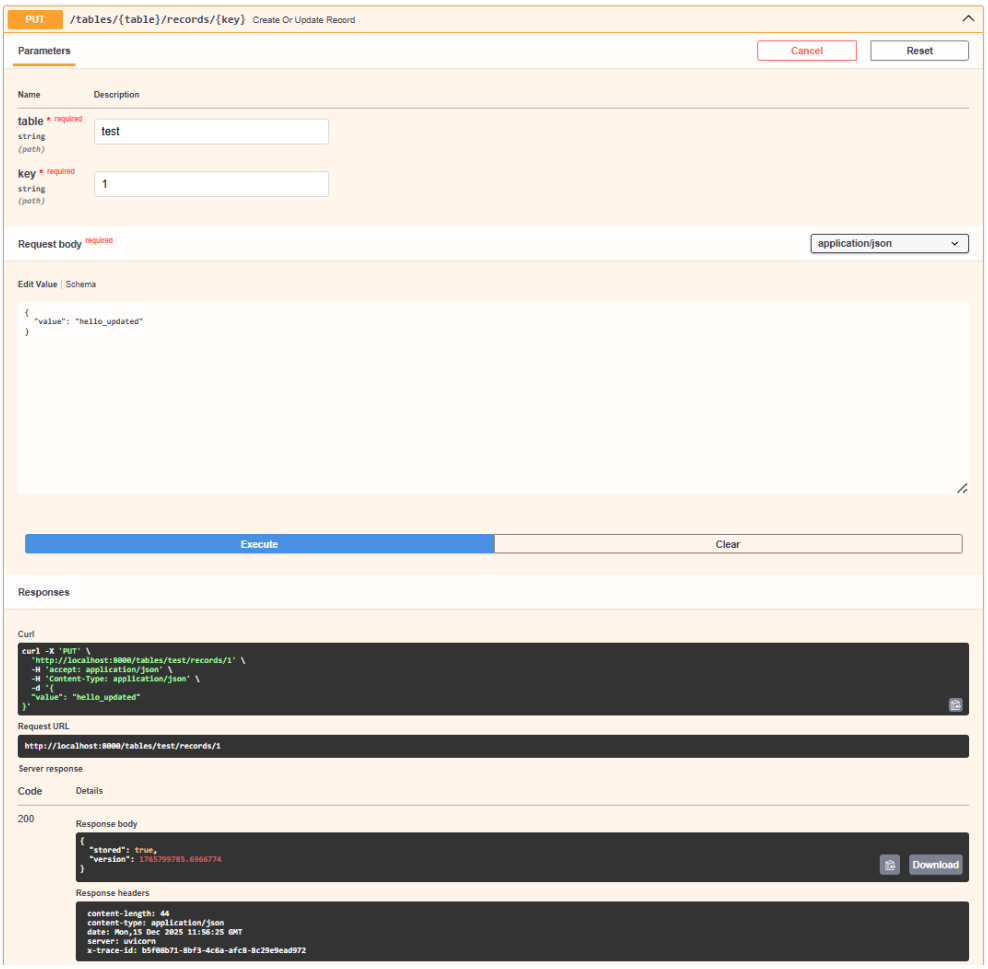
    # Last-Write-Wins logic
    if local is None or incoming["version"] > local["version"]:
        storage.setdefault(table, {})[key] = incoming
        last_applied_version = incoming["version"]
        save_offset(last_applied_version)

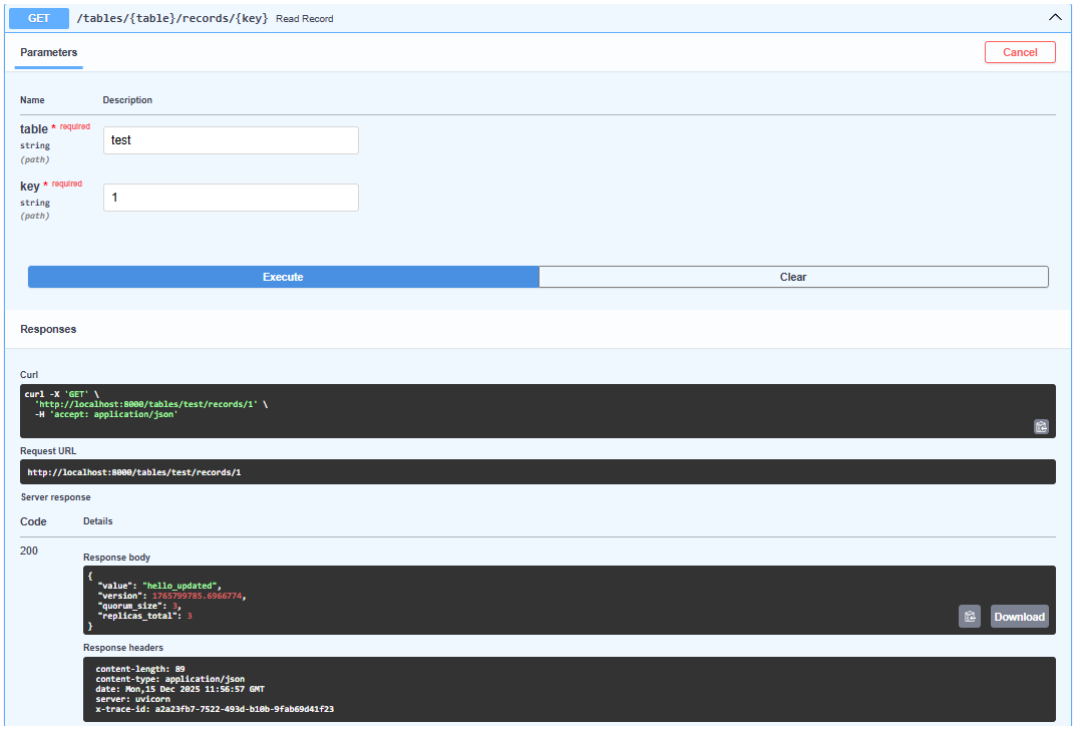
        return {
            "status": "applied",
            "version": incoming["version"]
        }

    return {
        "status": "ignored",
        "version": local["version"]
    }
```

Стратегія Last-Write-Wins реалізується під час реплікації даних. Якщо локальний запис відсутній, новий запис зберігається без перевірок. Якщо ж локальна версія існує, відбувається порівняння полів version, які містять часові мітки. Запис із більшою часовою міткою вважається новішим та перезаписує локальний, а запис із меншою версією ігнорується.





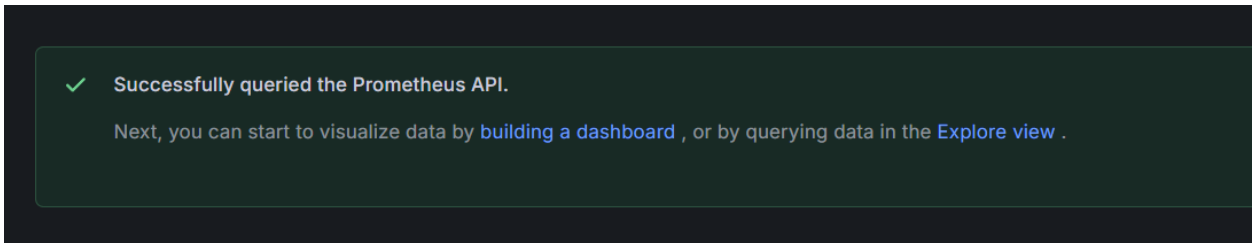


Для демонстрації роботи стратегії Last-Write-Wins було виконано декілька операцій запису для одного ключа. Кожен запис отримував часову мітку у полі version. Після виконання двох операцій PUT з різними значеннями

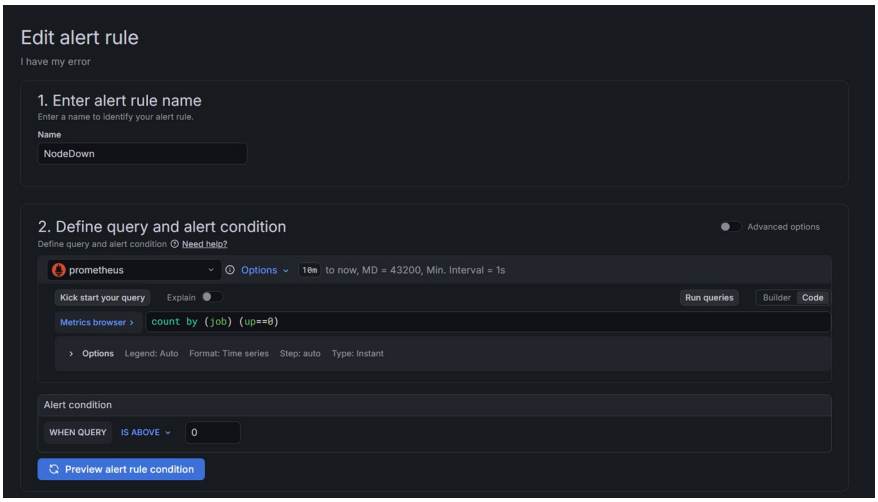
система зберегла запис з більшою часовою міткою. Під час операції GET було повернуто останню версію запису, що підтверджує коректну роботу стратегії LWW.

3. N8N Agentic Integration

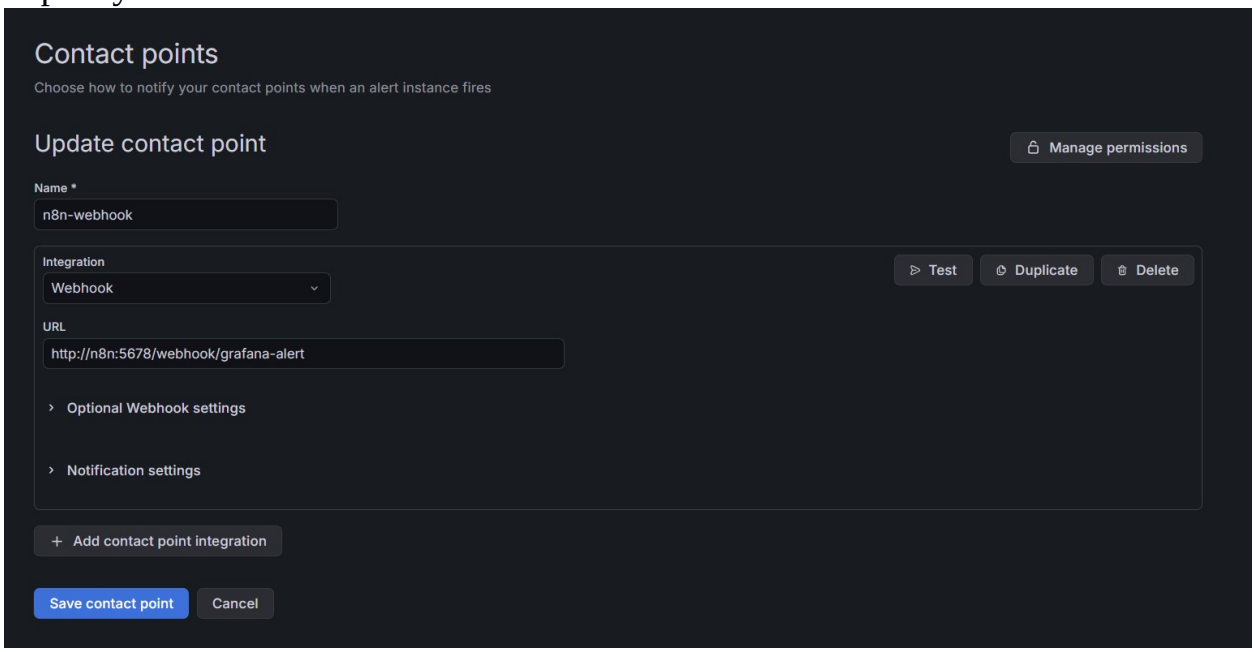
На даному етапі було реалізовано інтеграцію системи моніторингу Prometheus та Grafana з інструментом автоматизації n8n. Спочатку було перевірено коректність підключення Prometheus до Grafana, що підтверджується успішним виконанням запиту до Prometheus API та можливістю отримувати метрики для подальшої візуалізації і аналізу. Це забезпечує основу для побудови правил алертингу на основі стану сервісів і шард.



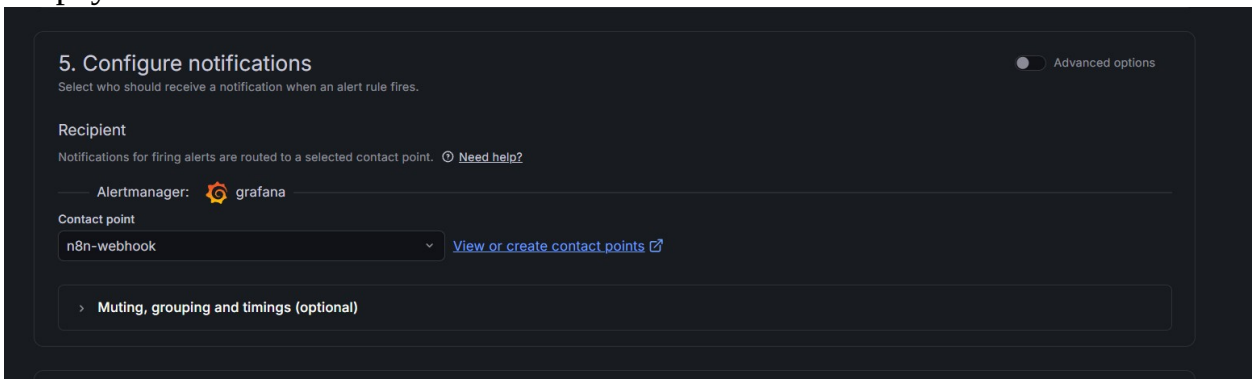
Далі в Grafana було створено правило алертингу NodeDown, яке призначене для виявлення відмови окремих сервісів або шард. Для цього використано PromQL-запит count by (job) (up == 0), який дозволяє визначити кількість сервісів, що перебувають у неактивному стані. Умову спрацювання алерту задано як перевищення нуля, тобто алерт активується у разі, якщо хоча б один сервіс стає недоступним. Результатом цього етапу є автоматичне визначення факту відмови компонента розподіленої системи.



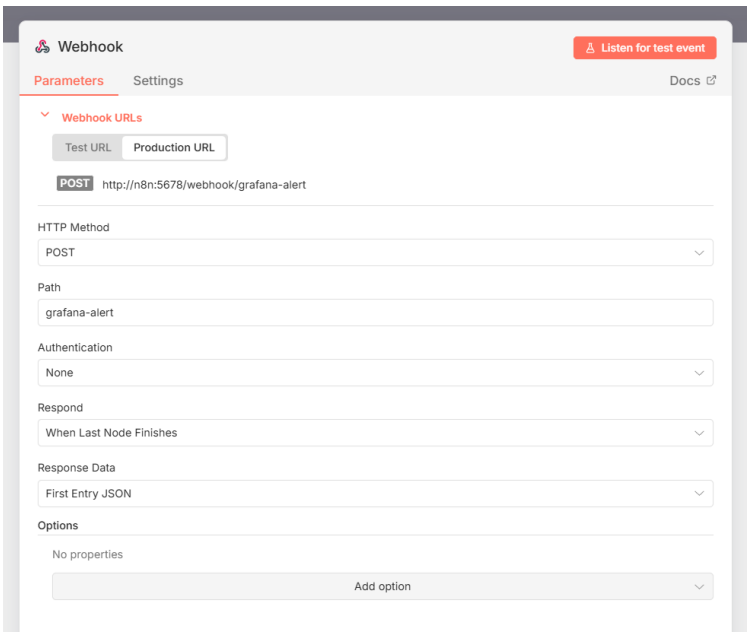
Після цього було налаштовано contact point типу Webhook у Grafana. В якості адреси для надсилання сповіщень використано webhook-ендпоінт сервісу n8n. Таким чином, Grafana при спрацюванні алерту надсилає HTTP POST-запит з інформацією про подію у зовнішню систему автоматизації. Це дозволяє інтегрувати алертинг з подальшими автоматичними діями без участі користувача.



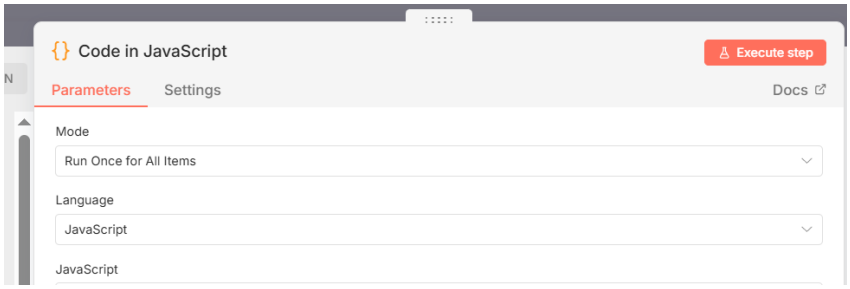
Після налаштування contact point треба вставити новий контакт поінт до алерту.



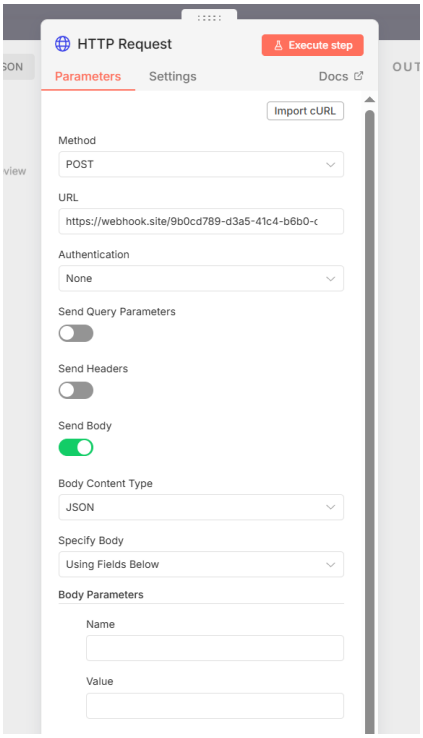
Наступним кроком було створення workflow у n8n, який починається з вузла Webhook. Цей вузол приймає HTTP POST-запити від Grafana та слугує точкою входу для всієї автоматизації. Webhook налаштований на прийом запитів за заданим шляхом, що забезпечує коректний зв’язок між Grafana та n8n. У результаті кожне спрацювання алерту ініціює виконання workflow.



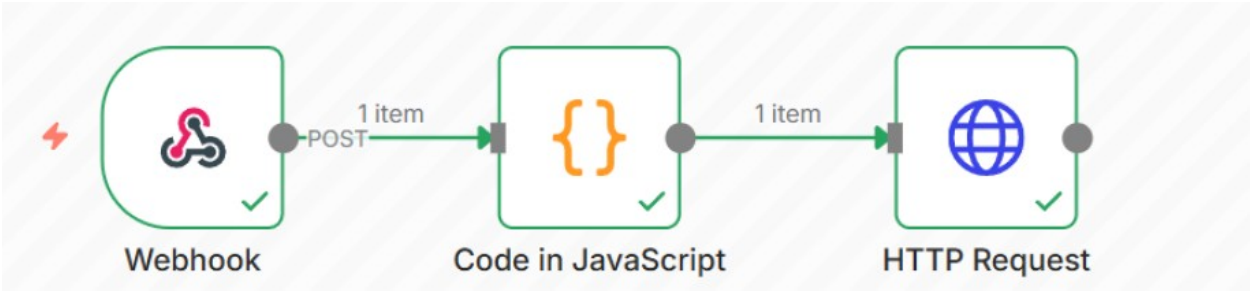
Після отримання даних алерту вони обробляються у вузлі Code in JavaScript. На цьому етапі відбувається аналіз і підготовка отриманого payload від Grafana у зручному для подальшої обробки форматі. Даний вузол дозволяє реалізувати власну логіку обробки подій, наприклад фільтрацію, агрегацію або трансформацію даних. Результатом є структуровані дані про інцидент, готові для передачі у зовнішні сервіси.



Завершальним етапом workflow є вузол HTTP Request, який виконує HTTP POST-запит до зовнішнього сервісу (наприклад, тестового endpoint або системи сповіщень). У запит передається JSON-повідомлення з інформацією про спрацювання алерту. Це демонструє можливість автоматичної реакції на інциденти, наприклад надсилання повідомлень, інтеграцію з іншими системами або запуск додаткових процесів.



У результаті виконання даного пункту реалізовано повноцінний ланцюг автоматичного реагування: від виявлення відмови сервісу у Prometheus і Grafana до обробки події та передачі її у зовнішні системи за допомогою n8n. Це підтверджує можливість використання n8n як агентної платформи для оркестрації реакцій на події у розподілених системах.



Workflow складається з трьох nodes:

Webhook node: приймає POST запити від Grafana на production URL

Code in JavaScript node: парсить Grafana payload, витягує alert_name, status, summary з body.alerts

HTTP Request node: надсилає форматоване повідомлення до webhook.site

Висновок.

У ході виконання лабораторної роботи було спроєктовано та реалізовано розподілену key-value систему з горизонтальним масштабуванням на основі шардингу та реплікації. Було реалізовано сервіс-координатор, який відповідає за маршрутизацію запитів і управління шардом, а також кілька шард-вузлів із підтримкою ролей лідера та реплік. Такий підхід дозволяє рівномірно розподіляти навантаження між вузлами та забезпечує відмовостійкість системи у разі збоїв окремих компонентів.

У роботі було впроваджено механізми версіонування записів і стратегію вирішення конфліктів Last-Write-Wins, що дозволяє коректно обробляти конкурентні оновлення даних у розподіленому середовищі. Також було інтегровано систему моніторингу на базі Prometheus і Grafana з налаштуванням метрик та алертів для контролю доступності вузлів, затримок запитів і стану реплікації. Окрему увагу приділено автоматизації операційних процесів із використанням n8n, зокрема реалізовано автоматичні періодичні задачі для збору стану системи та формування звітів, що зменшує потребу в ручному адмініструванні.

Таким чином, у результаті виконання роботи було створено функціональну прототипну розподілену систему з елементами спостережуваності, автоматизованого реагування та операційної автоматизації. Отримані результати демонструють практичне застосування принципів побудови розподілених систем, моніторингу та автоматизації інцидентів, а також підтверджують доцільність використання сучасних інструментів для забезпечення надійності та керованості сервісів.